INTRODUCTION

PROJECT TITLE : AI Weather Wise API

TEAM LEADER :

- **Brindha K & brindhakavimani22@gmail.com**

TEAM MEMBERS :

- **Egavarshini B & egavarshini2020@gmail.com**
- **Afrin Begam R & afrinbegum023@gmail.com**

2. PROJECT OVERVIEW :

AI WeatherWise is a robust RESTful backend application developed using Node.js and Express.js. It uses MongoDB with Mongoose for secure and efficient data storage. The system integrates real-time weather data with Google Gemini AI to generate easy-to-understand weather summaries and personalized recommendations based on current environmental conditions. It also provides JWT-based authentication, favorite location management, centralized error handling, and fallback support when external API services are unavailable.

Project Purpose :

The main purpose of AI WeatherWise is to provide users with a simple and intelligent platform for accessing real-time weather information and receiving personalized suggestions. Instead of displaying only raw weather measurements, the system uses AI to explain weather conditions in natural language and recommend suitable activities, clothing, and precautions. It also securely stores user preferences and favorite locations, allowing users to manage their weather information efficiently across devices.

Key Features / Modules :

User Authentication and Authorization :

- User registration and login

- JWT-based authentication
- Secure password handling

Real-Time Weather Information :

- Fetches current weather data from external weather APIs
- Provides temperature, humidity, wind speed, pressure
- Supports weather information based on city or location

AI-Powered Weather Summaries :

- Integrates Google Gemini AI
- Converts complex weather data into simple, easy-to-understand summaries
- Provides personalized recommendations based on current weather conditions

Personalized Recommendations :

- Suggests suitable activities based on the weather
- Recommends appropriate clothing
- Provides safety precautions for extreme weather conditions

Favorite Location Management :

- Allows users to add and remove favorite locations

- Enables quick access to weather information for saved locations
- Stores user preferences securely in MongoDB

RESTful API Modules :

- User authentication APIs
- Weather data APIs
- AI recommendation APIs
- Favorite location APIs
- User profile and preference APIs

Database Management :

- Uses MongoDB for data storage
- Uses Mongoose for schema definition and database operations
- Stores user accounts, preferences, favorite locations, and related data securely

Centralized Error Handling :

- Provides consistent error responses across the application
- Handles validation errors, authentication errors, database errors, and API fail
- Improves debugging and application reliability

External API Integration :

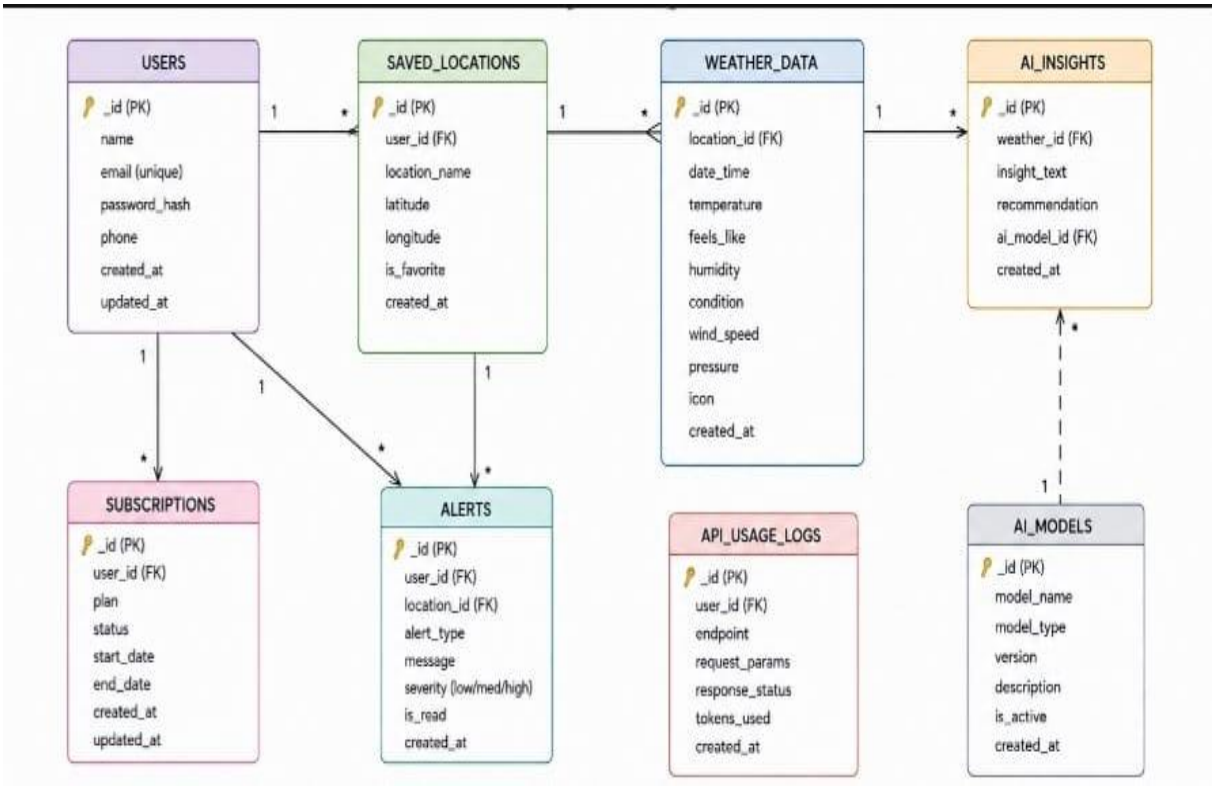
- Integrates weather service APIs for real-time data

- Integrates Google Gemini AI for intelligent analysis

- Handles external service communication efficiently

3. ARHITECTURE

The AI WeatherWise application follows the MVC pattern to decouple business logic from data management. The Controller layer orchestrates weather fetching from OpenWeatherMap and generation of AI insights via Gemini, while the Model layer manages User and Location entities. Express Server Gateway: Monitors network ports, maps explicit HTTP methods (GET, POST, PUT, DELETE), parses JSON payloads, and handles CORS headers. Authentication & Role Matrix Middleware: Extracts tokens from headers, validates user context signatures, and evaluates if the specific role satisfies authorization thresholds for guarded resource paths. Granular Services (Logic Modules): Enforces isolated domain rules for blog lifecycles (Draft, Pending Approval, Scheduled, Published), comment queues, and system reporting indicators. Database Interface Layer: Mongoose translates operational application logic into performant native MongoDB queries.



User Entity :

- Email (String, Required, Unique)
- Password (String, Hashed, Required)
- Role (String, Default: "reader")

Location Entity :

- _id (Primary Key - ObjectId)
- User (Foreign Key referencing User Collection)
- Ciity (String, Required)
- Country (String, Required)
- CreatedAt (Date, Default: Now)

Key Features :

- Role-Based Route Guarding: Strict token processing blocks unauthorized tiers from modifying core settings.

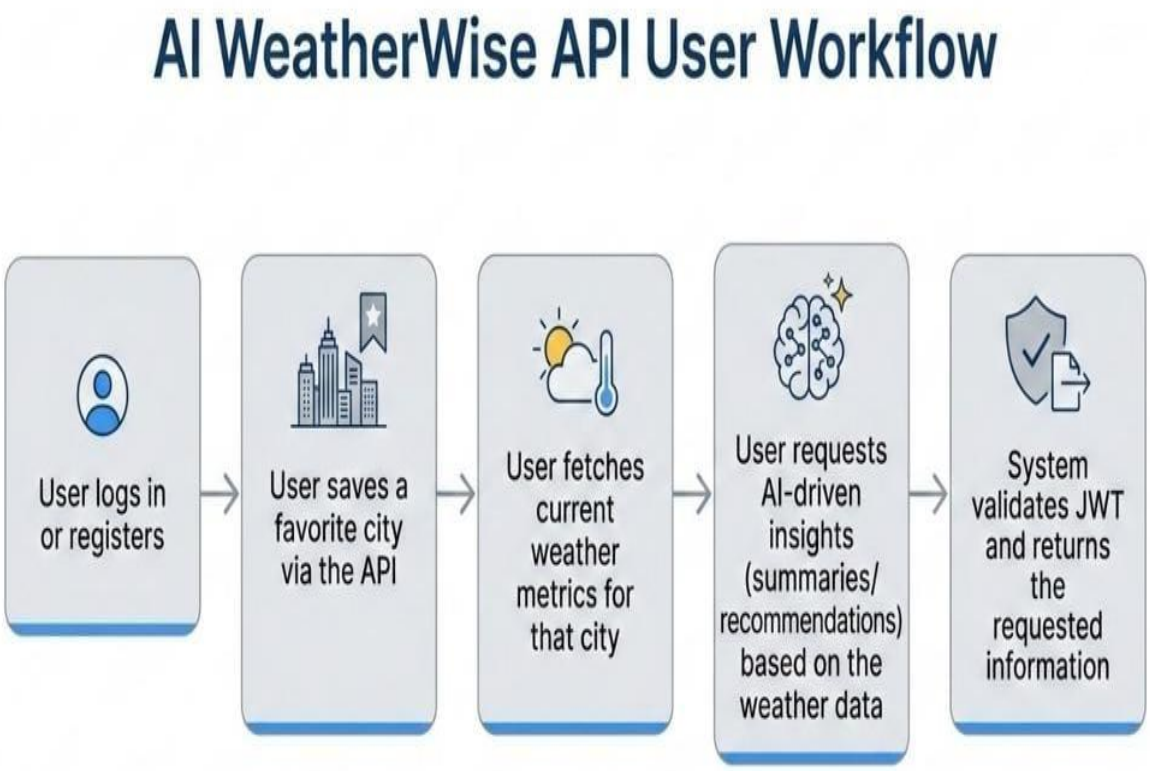
- CRUD Blog Execution Engine: Complete schema management over content definitions, status states, and asset assignment paths.

- Categorization System: Native tag indices facilitate filtering and querying for enhanced discoverability.

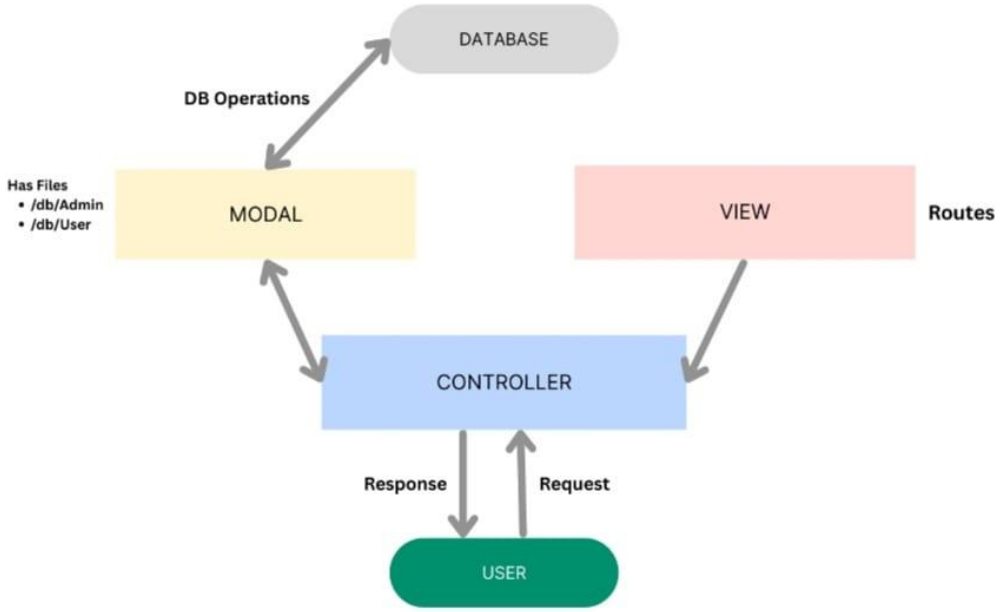
● Comment Moderation Logic: Protected entry points for updating moderation flags (approved, pending, spam).

User Flow :

1. User logs in or registers.
2. User saves a favorite city via the API.
3. User fetches current weather metrics for that city.
4. User requests AI-driven insights (summaries/recommendations) based on the weather data.
5. System validates JWT and returns the requested information.



MVC Architecture Pattern :



Model Layer (Data Blueprint) :

Defines structure blueprints via strongly enforced Mongoose schemas mapping explicit fields into raw collection items inside MongoDB. It sets default states, indexing rules, and transformation parameters before validation persistence occurs.

Controller Layer (Execution Matrix) :

The intermediary brain of the ecosystem. It captures request vectors from the execution routes, checks structural request constraints, delegates processing workflows directly to models, and packages the resultant output into standard JSON return arrays.

4. SETUP INSTRUCTIONS

Creating the Project Folder :

- Create a new folder named AI Weatherwise
- Open the project folder in Visual Studio Code.
- Initialize the project using Node.js and install the required dependencies.

Project Initialization

Open the project folder in the VS Code terminal and execute:

npm init -y

Install the required dependencies:

npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai

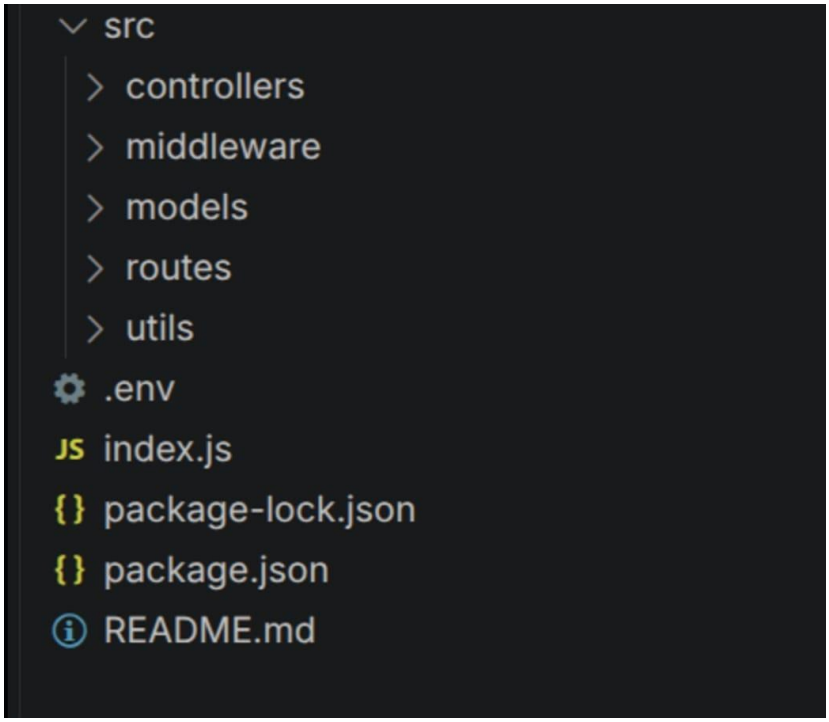
Install Nodemon for development:

npm install --save-dev nodemon

Create the Following Files

- index.js
- .env
- package.json

5. Folder Structure



Environment Configuration :

Create a .env file in the project root and configure the required environment variables.

PORT=5000

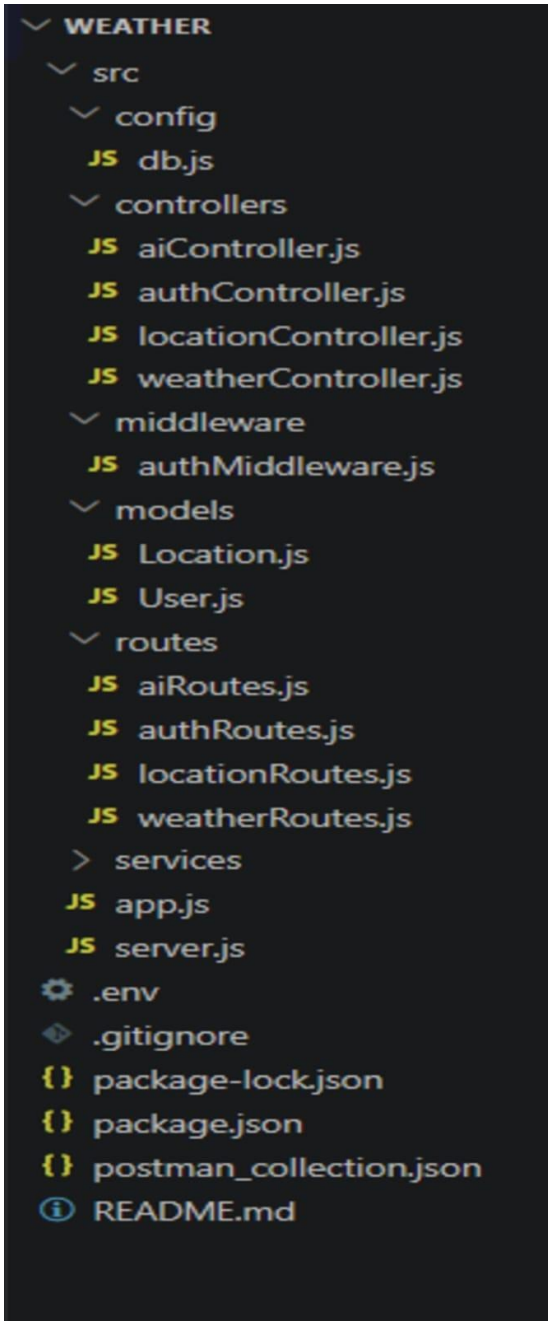
MONGO_URI=your_mongodb_connection_string

JWT_SECRET=your_jwt_secret

GEMINI_API_KEY=your_gemini_api_key

Backend Development :

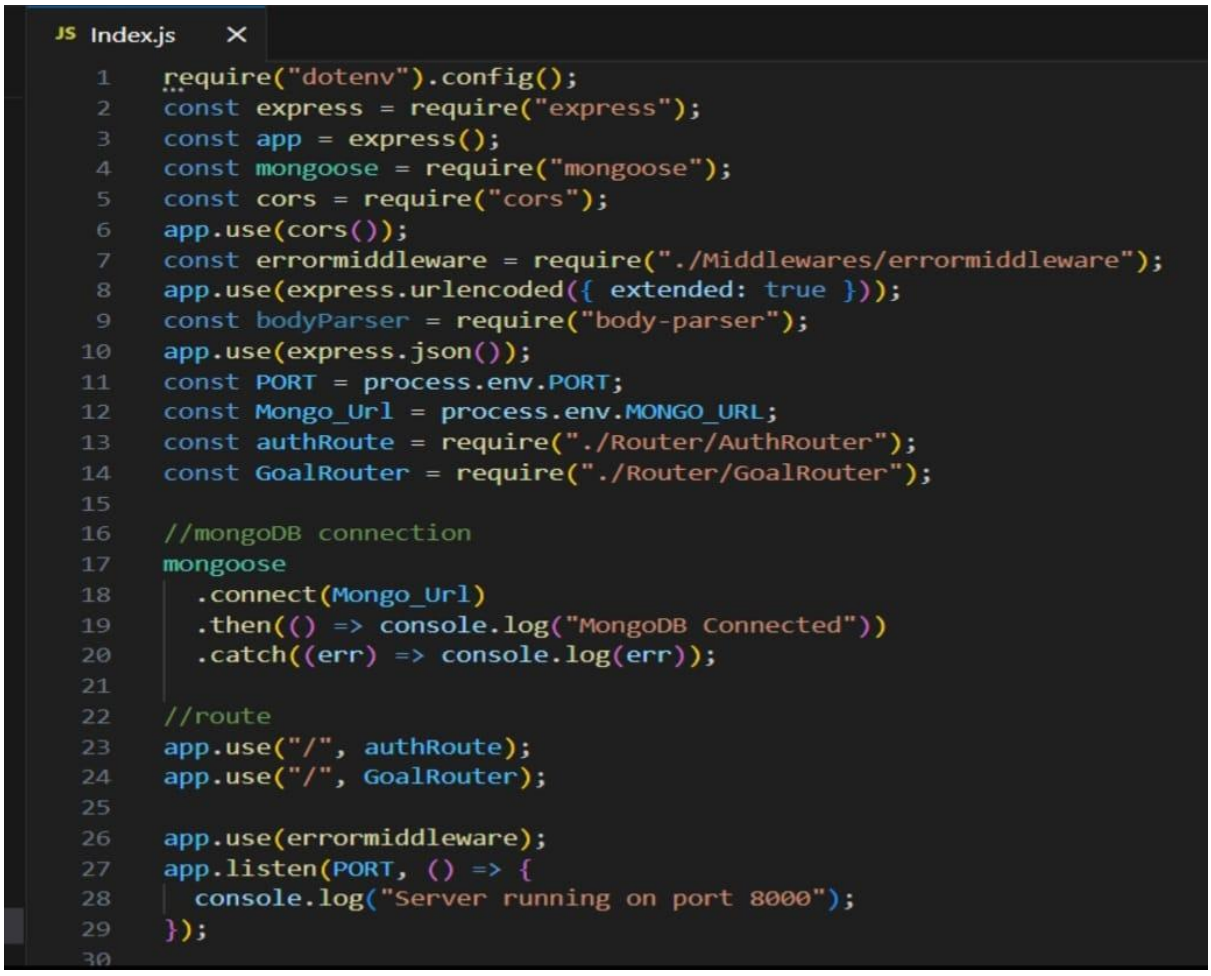
The AI weatherwise backend is developed using Node.js, Express.js, and MongoDB, following a modular architecture. The application separates routing, controllers, middleware, models, and utility modules to improve code maintainability and scalability. The backend exposes secure REST APIs for authentication, study material management, AI-powered content generation, and administrative operations.



6. RUNNING THE APPLICATION

Created Express Server :

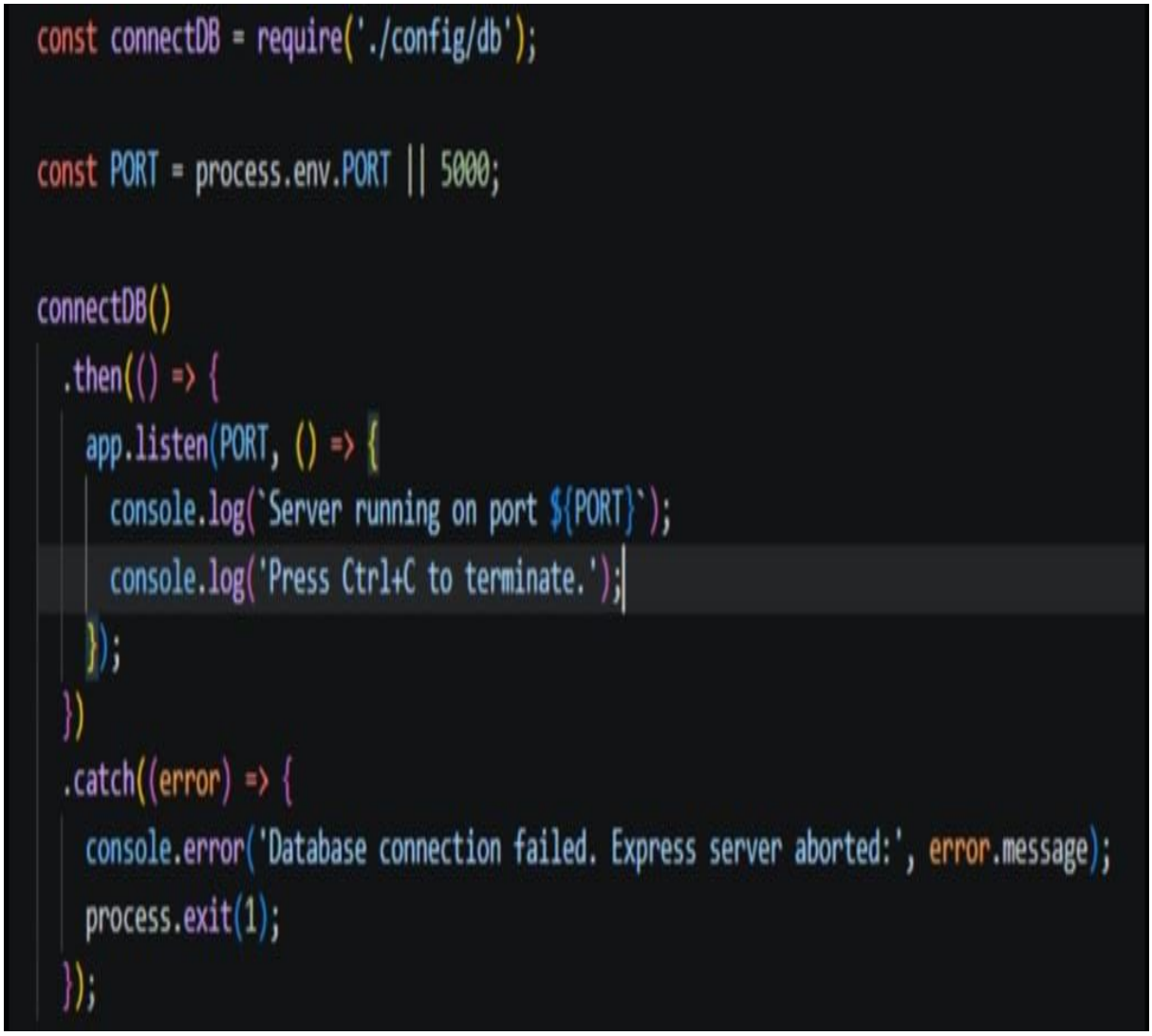
An Express server was set up to manage and handle API requests from the client side. Express is a popular Node.js framework used for building backend web applications and APIs efficiently. The server was configured to connect different routes that define how the application responds to various requests such as GET, POST, PUT, and DELETE. JSON parsing middleware was also enabled so that the server can easily read and process data sent in JSON format. This helps in handling user inputs and transferring data between the client and the database.



7. API DOCUMENTATION

Application Entry Point (index.js) ;

The index.js file serves as the main entry point of the application. It initializes the Express server, loads environment variables, configures middleware, establishes the MongoDB connection, registers API routes, and starts the server after a successful database connection.



```
const connectDB = require('./config/db');

const PORT = process.env.PORT || 5000;

connectDB()
  .then(() => {
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
      console.log('Press Ctrl+C to terminate.');
    });
  })
  .catch((error) => {
    console.error('Database connection failed. Express server aborted:', error.message);
    process.exit(1);
  });
```

8. AUTHENTICATION

Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below.

Mongoose Schema Definitions

Location Schema:

```
const mongoose = require('mongoose');

const locationSchema = new mongoose.Schema({
  city: {
    type: String,
    required: [true, 'Please provide a city name'],
    trim: true
  },
  country: {
    type: String,
    required: [true, 'Please provide a country name'],
    trim: true
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: [true, 'User reference is required']
  }
}, {
  timestamps: true
});

// Ensure a user cannot duplicate the exact same city and country in their favorites
locationSchema.index({ user: 1, city: 1, country: 1 }, { unique: true });

const Location = mongoose.model('Location', locationSchema);
module.exports = Location;
```

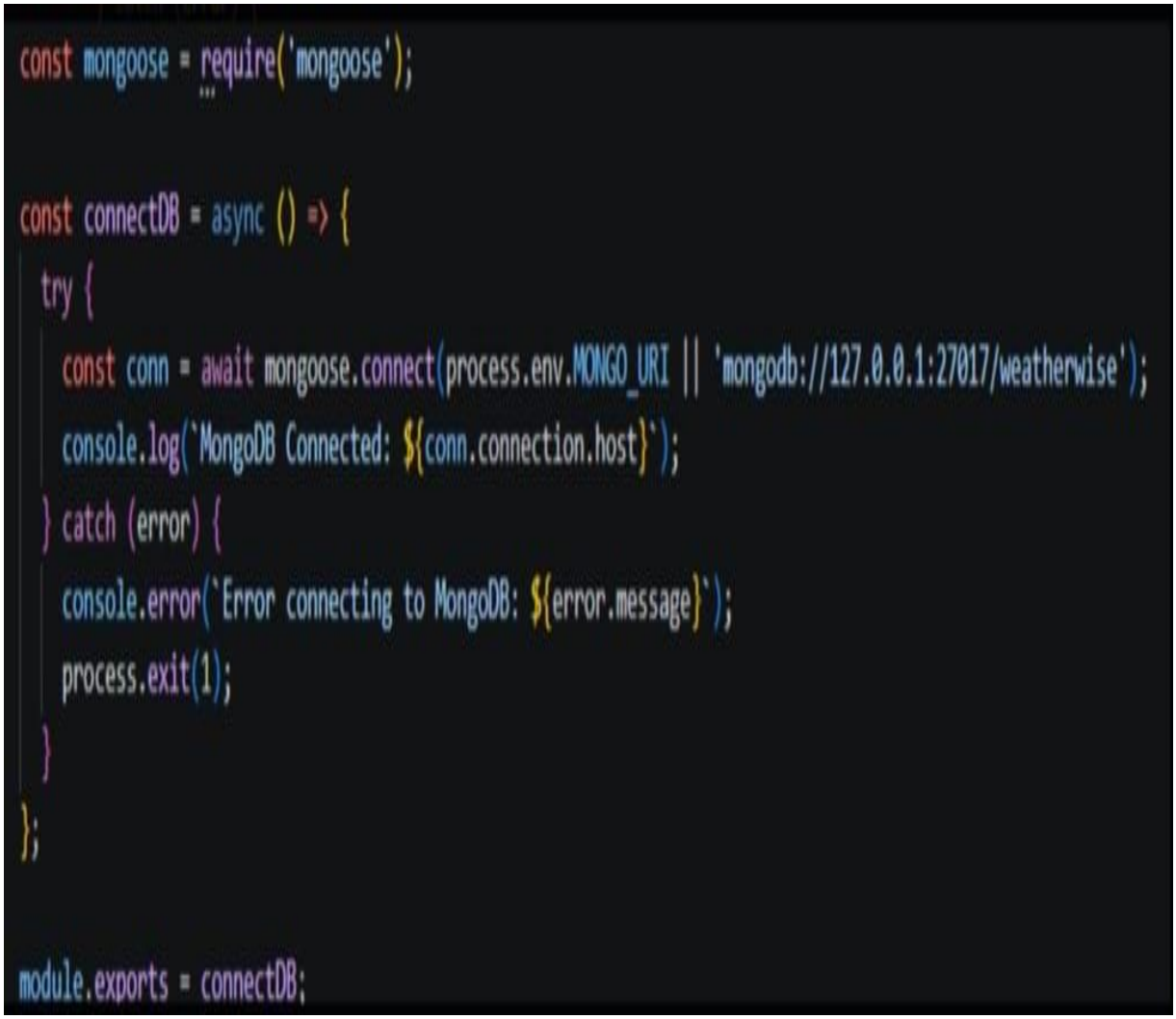
User Schema :

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please provide a name'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Please provide an email'],
    unique: true,
    trim: true,
    lowercase: true,
    match: [
      /^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
      'Please provide a valid email address'
    ]
  },
  password: {
    type: String,
    required: [true, 'Please provide a password'],
    minlength: [6, 'Password must be at least 6 characters']
  }
}, {
  timestamps: true
});
```

Db connection:

This module establishes a connection between the application and MongoDB using Mongoose. It ensures that the backend communicates with the database before processing any client requests.



```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/weatherwise');
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

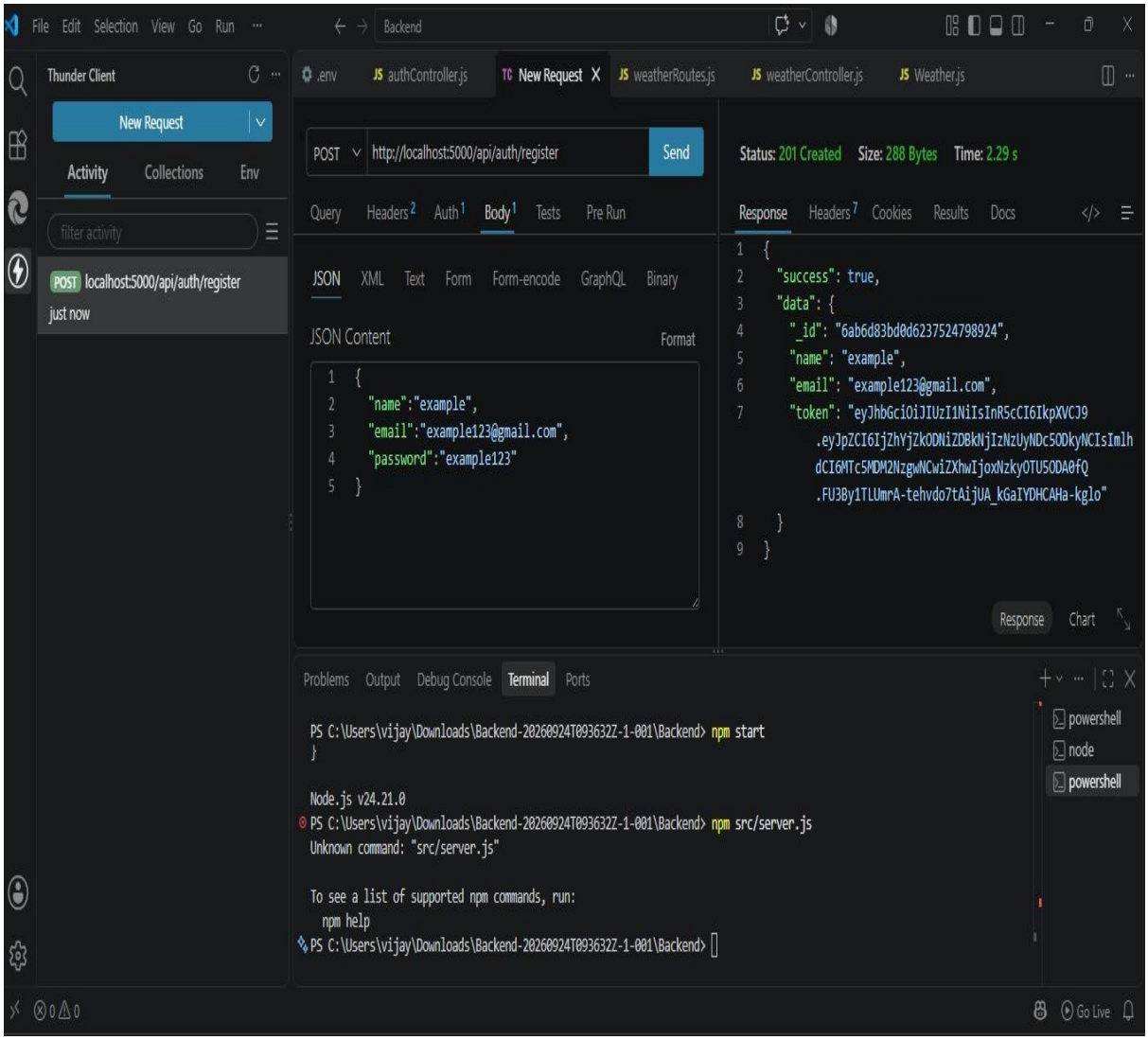
module.exports = connectDB;
```

9. OUTPUT SCREENSHOT

Register API :

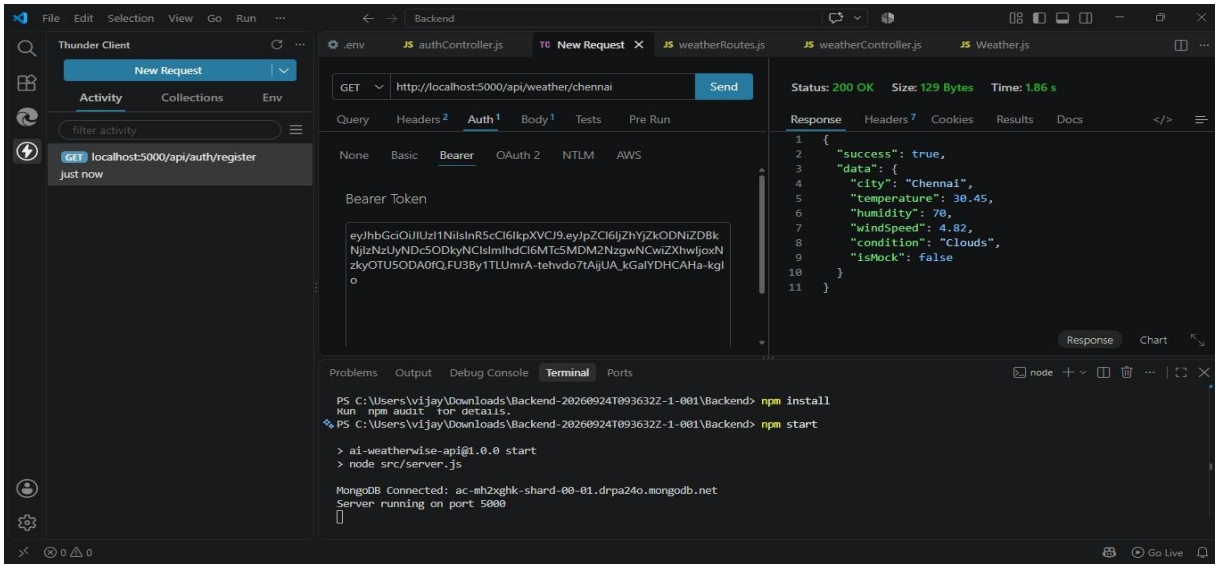
• Successful User Registration(Signup/Login)

• In this Api call the user register or login and token was generated



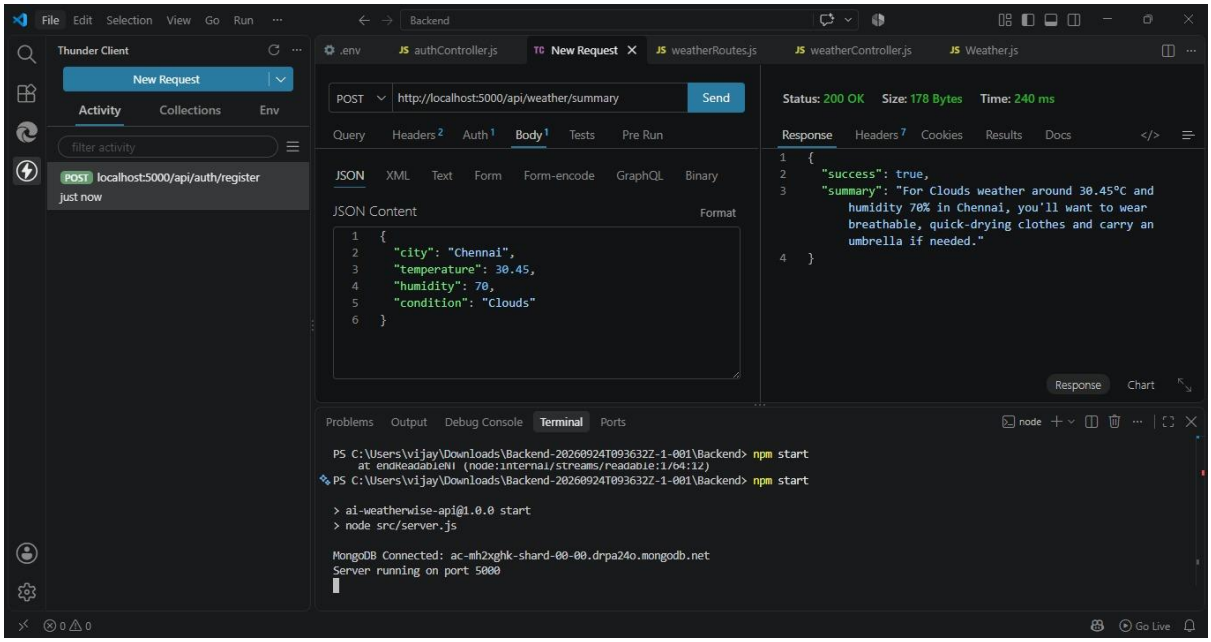
Login API :

After successful login user the api generate the token and these token was sent with every request to every api.



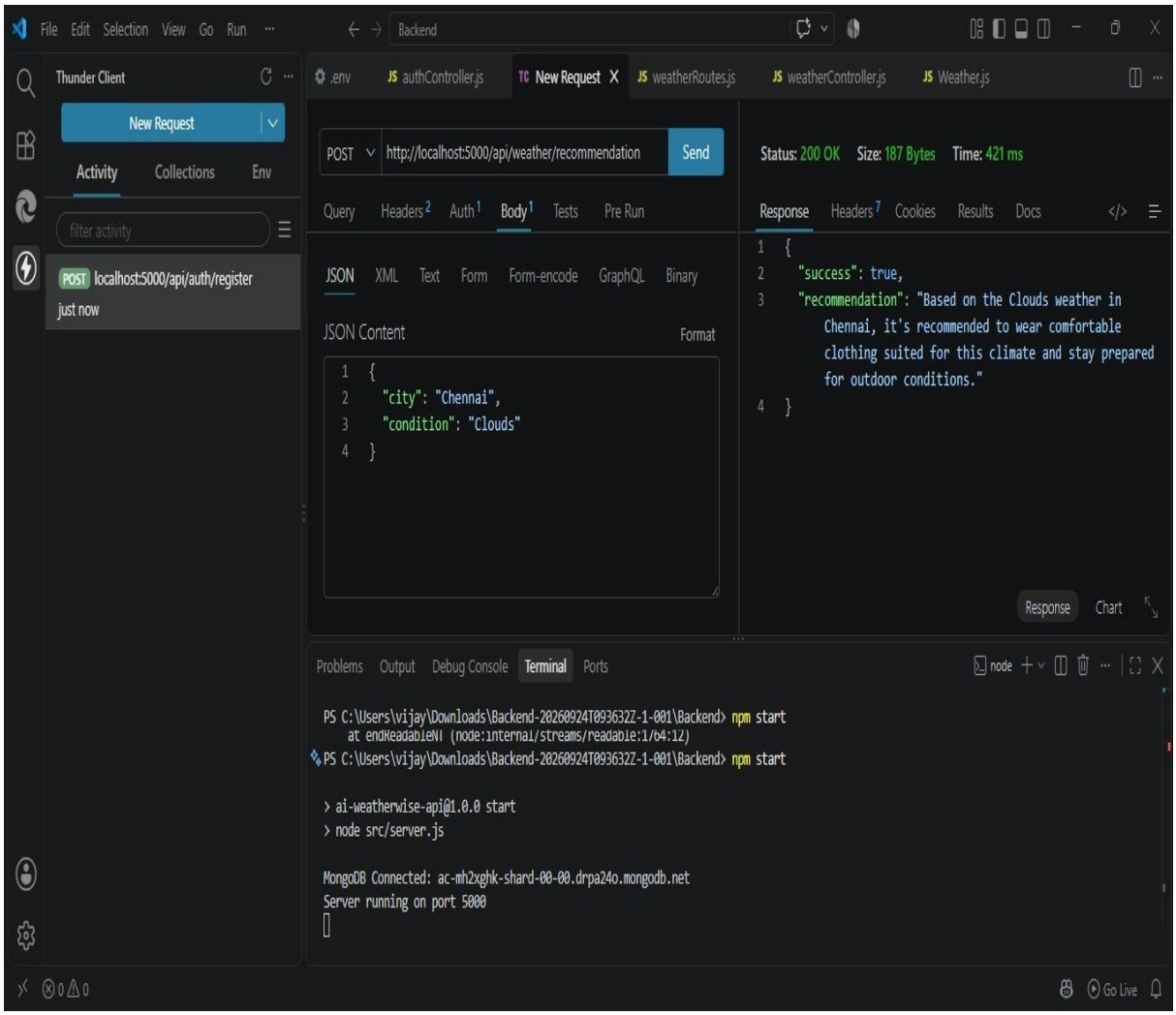
Weather API :

Weather API: Weather API is a service used to provide real-time weather information for a specific location.



Recommentation API :

Recommendation API analyzes the current weather conditions and provides suitable suggestions to the user. It can recommend activities, clothing, and safety precautions based on the weather data. In AI WeatherWise, Google Gemini AI is used to generate personalized recommendations.



CONCLUSION :

The AI WeatherWise project provides an intelligent and reliable platform for accessing and understanding real-time weather information. The system combines Node.js, Express.js, MongoDB, Weather API, and Google Gemini AI to deliver accurate weather details along with useful personalized recommendations.

The application allows users to securely manage their accounts and favorite locations while easily accessing weather information such as temperature, humidity, wind speed, and weather conditions. The integration of Gemini AI further improves the system by converting raw weather data into simple summaries and suggesting suitable activities, clothing, and safety precautions.

With JWT authentication, secure password handling, input validation, centralized error handling, and fallback support, the system provides better security, reliability, and usability. Overall, AI WeatherWise demonstrates how modern backend technologies and artificial intelligence can be combined to create a practical and user-friendly weather application.